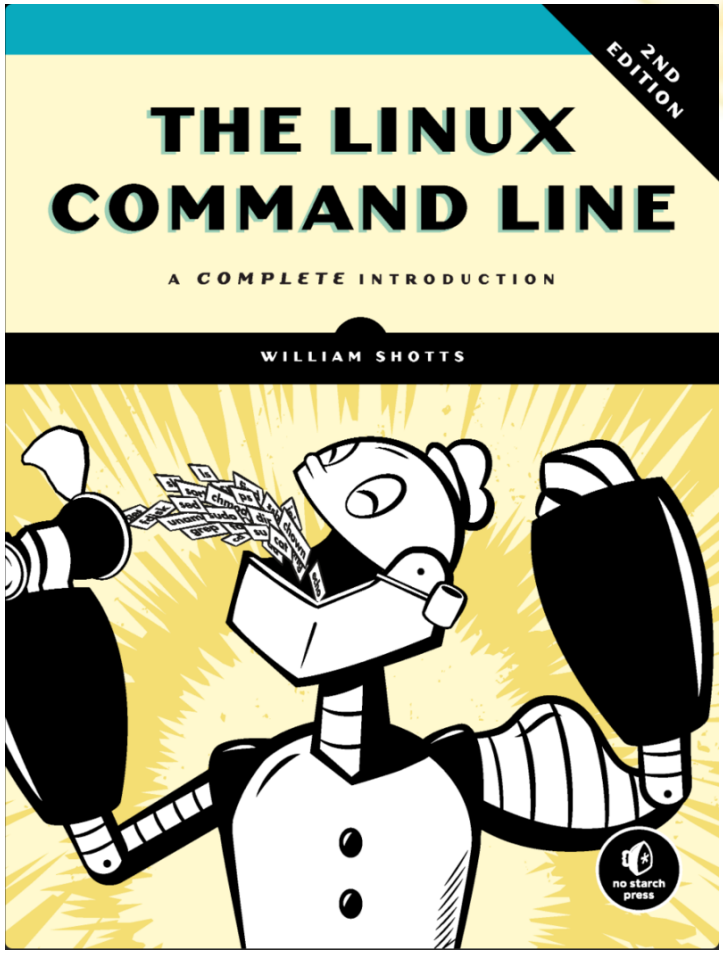
UMD DATA605 - Big Data Systems

02.1: Git

Instructor: Dr. GP Saggese, gsaggese@umd.edu

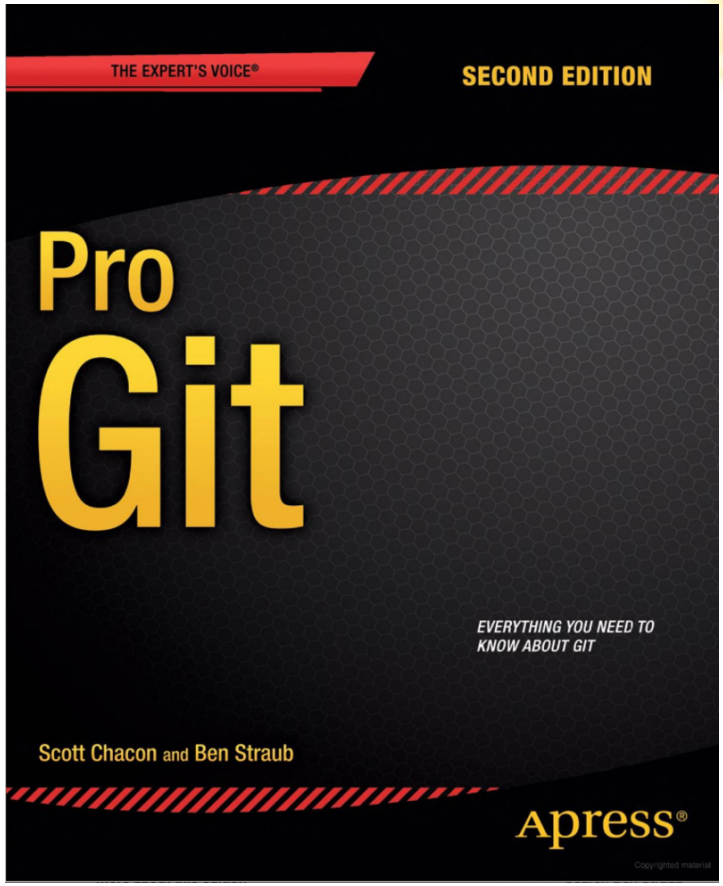
Bash / Linux: Resources

- **How Linux works**
  - Processes
  - File ownership and permissions
  - Virtual memory
  - How to administer a Linux box as root
- **Easy**
  - Command-Line for Beginners
  - E.g., `find`, `xargs`, `chmod`, `chown`, symbolic, and hard links
- **Mastery**
  - The Linux Command Line



Git Resources

- Concepts in the slides
- Tutorial: Tutorial Git
- We will use Git during the project
- Mastery: Pro Git (free)
- Web resources:
  - https://githowto.com
  - dangitgit.com (without swearing)
  - Oh Sh*t, Git!?! (with swearing)
- Playgrounds
  - https://learngitbranching.js.org

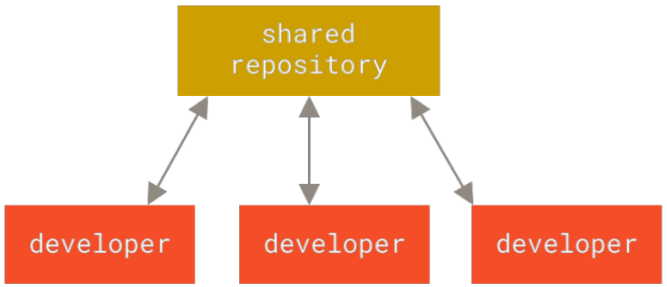


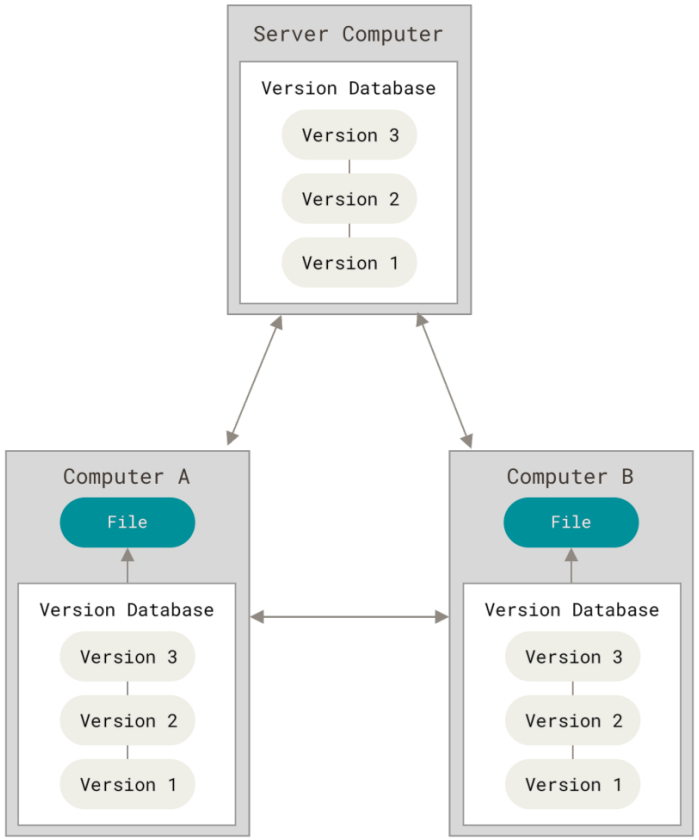
Version Control Systems (1/2)

- A **Version Control System** (VCS) is a system that allows to:
  - Record changes to files
  - Recall specific versions later (like a "file time-machine")
  - Compare changes to files over time
  - Track *who* changed *what* and *when* and *why*
- **Simplest "VCS"**
  - Make a copy of a dir and add
    - `_v1` (bad); or
    - a timestamp `_20220101` (better)
  - **Cons**: It kind of works for one person, but doesn't scale

Version Control Systems (2/2)

- **Centralized VCS**
  - E.g., `Perforce`, `Subversion`
  - A server stores the code, clients connect to it
  - **Cons**: If the server is down, nobody can work
- **Distributed VCS**
  - E.g., `Git`, `Mercurial`, `Bazaar`, `Dart`
  - Each client has the entire history of the repo locally
  - Each node is both a client and a server
  - **Cons**: complex

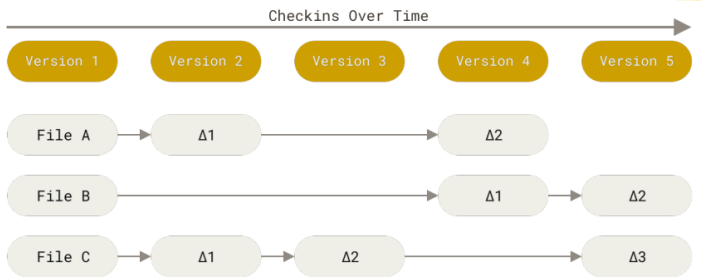




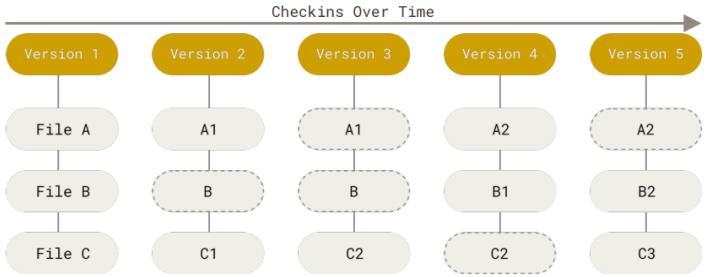
SCIENCE ACADEMY

VCS: How to Track Data

- Consider a directory with project files inside
- **How do you track changes to the data?**
- **Delta-based VCS**
  - E.g., `Subversion`
  - Store the data in terms of patches (changes of files over time)
  - Can reconstruct the state of the repo by applying the patches



- **Stream of snapshots VCS**
  - E.g., `Git`
  - Store data in terms of snapshots of a filesystem
  - Take a "picture" of what files look like
  - Store reference (hash) to the snapshots
  - Save link to previous identical files

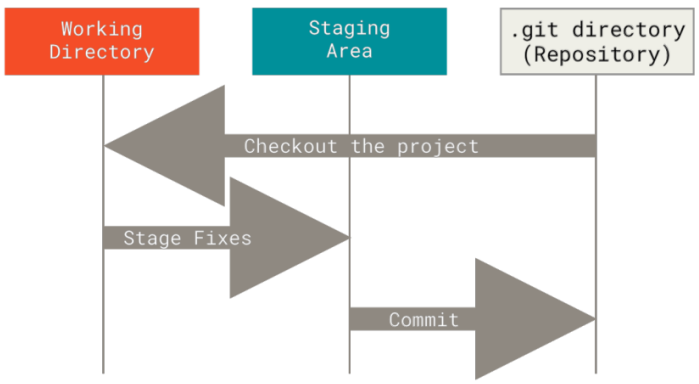


Git

- **Almost everything is local for Git**
  - History stored locally in each node
  - Diff-ing files done locally
    - Centralized VCS requires server access
  - Commit to local copy
    - Upload changes with network connection
- **Almost everything is undoable in Git**
  - No data corruption
    - Everything checksummed
  - Nothing lost
  - Disclaimer:
    - Commit (at least locally) or stash
    - Know how to do it to avoid "git hell"
- **Git is a mini key-value store with a VCS built on top**
  - Exactly true
  - Two layers:
    - "porcelain": key-value store for file-system
    - "plumbing": VCS layer

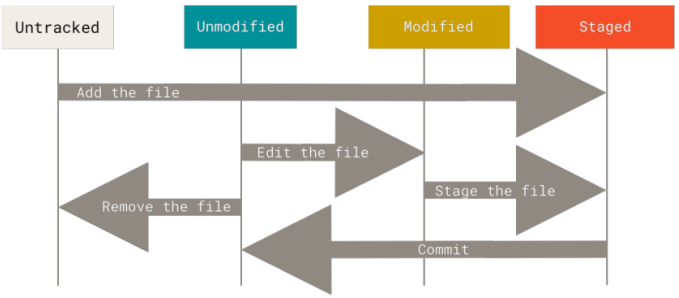
Sections of a Git Project

- There are 3 main sections of a Git project
  - **Working tree** (aka checkout)
    - Version of code on the filesystem for use and modification
  - **Staging area** (aka cache, index)
    - File in `.git` storing info for the next commit
  - **Git directory** (aka .git)
    - Stores metadata and objects (like a DB)
    - The repo itself with all history
    - Cloning gets you the project's `.git`



States of a File in Git

- Each file can be in 4 states from Git point-of-view
  - **Untracked**: files not under Git version control
  - **Modified**: changed files, not committed yet
  - **Staged**: marked modified files for next commit
  - **Committed**: data stored in local DB



Git Tutorial

- **Git tutorial on class repo**
  - Follow the `README`
- **How to use a tutorial**
  - Type commands one-by-one
    - Avoid copy-paste
  - Observe results
    - Understand each line
  - Experiment
    - *"What happens if I do this?"*
    - *"Does the result match my mental model?"*
  - Learn command line before GUI
    - GUIs hide details and you become dependent on it
- Go through **recommended Git book** and try all examples
  - Use online tutorials
- Build your own **cheat sheet**
  - Reuse others' cheat sheets only if familiar
- Achieve **mastery of basic tools**
  - Bash, Git, editor
  - Python, Pandas
  - ...

SCIENCE ACADEMY

Git Daily Use

- Check out a project (`git clone`) or start from scratch (`git init`)
  - Only once per Git project client
- **Daily routine**
  - Modify files in working tree (`vi ...`)
  - Add files (`git add ...`)
  - Stage changes for next commit (`git add -u ...`)
  - Commit changes to `.git` (`git commit`)
- **Use a branch to group commits together**
  - Isolate code from changes in `master`
  - Merge `master` into branch
  - Isolate `master` from changes
  - Pull Request (PR) for code review
  - Merge PR into upstream

Git Remote

- **Remote repos**: versions of the project hosted online
  - Manage remote repos to collaborate
  - Push/pull changes
  - `git remote -v`: show remotes
  - `git fetch`: pull data from remote repo you don't have locally
  - `git pull`: shorthand for `git fetch origin` + `git merge master --rebase`
  - `git push <REMOTE> <BRANCH>`: push local data to remote
    - E.g., `git push origin master`
- Multiple forks of the same repo with different policies
  - E.g., read-only, read-write
- If someone pushed to remote, you **can't push changes immediately**:
  - Fetch changes
  - Merge changes to your branch
  - Resolve conflicts, if needed
  - Test project sanity (e.g., run unit tests)
  - Push changes to remote

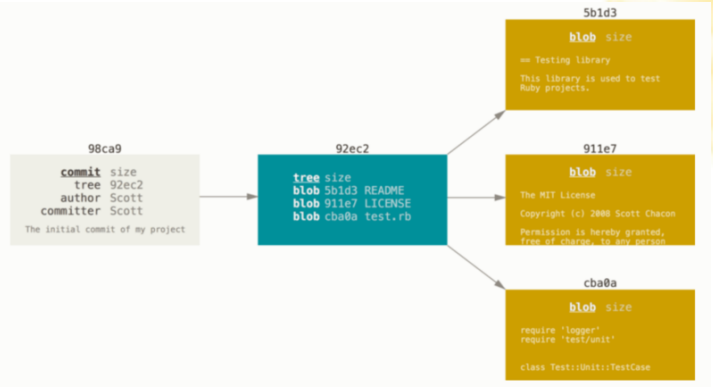
Git Tagging

- Git allows marking points in history with a **tag**
  - E.g., release points
  - Check out a tag
- Enter detached `HEAD` state
  - Committing won't add changes to the tag or branch
  - Commit will be "unreachable," reachable only by commit hash

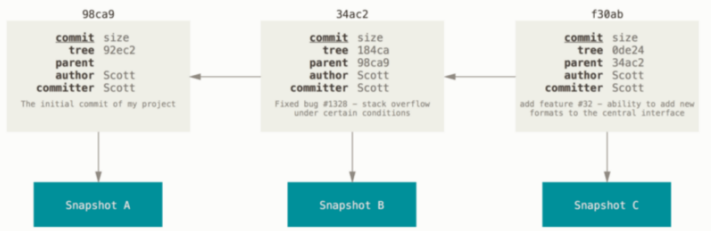
Git Internals

- **Understand Git only if you understand its data model**
  - Git is a key-value store with a VCS interface
  - Key = hash of a file
  - Value = content of a file
- **Git objects**
  - Commits: pointers to the tree and commit metadata
  - Trees: directories and mapping between files and blobs
  - Blobs: content of files
- Refs:
  - Easy: Understanding Git Data Model
  - Hard-core: Git internals

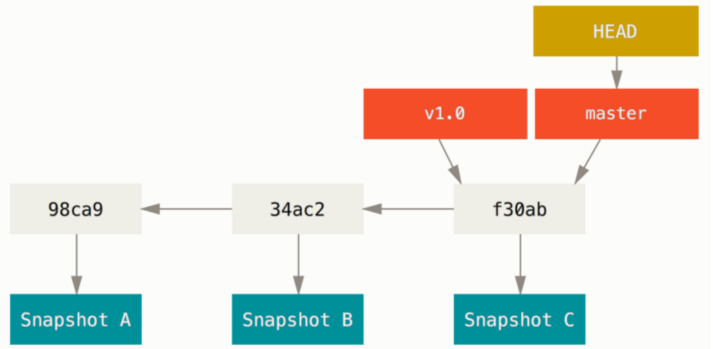
Commit tree



Commit parents



Commit history of a branch



SCIENCE ACADEMY

GitHub

- GitHub acquired by MSFT for $7.5 billion
- **GitHub: largest host for Git repositories**
  - Git hosting (100M+ open source projects)
  - Pull Requests (PRs), forks
  - Issue tracking
  - Code review
  - Collaboration
  - Wiki
  - Actions (CI/CD)
- **"Forking a project"**
  - Open-source communities
    - Negative connotation
    - Modify and create a competing project
  - GitHub parlance
    - Copy a project to contribute without push/write access